

ESEMPI DI MAPPE

TIPOLOGIA 1: IL Dizionario Contatore (Valori Numerici)

Struttura: { ID_NODO : Numero } (es. { 5: 12 } -> Il brano 5 ha venduto 12 copie).

Quando usarlo: **Quando la regola per creare l'arco si basa su quantità, somme, medie o conteggi.**

Possibili richieste all'esame:

1. "Due Nodi sono collegati se la differenza del numero di vendite è inferiore a K".
 - SQL: SELECT TrackId, COUNT(InvoiceId) FROM... GROUP BY TrackId
2. "Collega due Nodi se i ricavi totali (UnitPrice * Quantity) sono uguali".
 - SQL: SELECT TrackId, SUM(UnitPrice * Quantity) FROM... GROUP BY TrackId
3. "Grafo diretto: L'arco va dall'artista A all'artista B se A ha prodotto più canzoni di B".
 - SQL: SELECT ArtistId, COUNT(TrackId) FROM... GROUP BY ArtistId
4. "L'arco tra due Dipendenti si crea se la differenza di fatturato dei loro clienti è minima".
 - SQL: SELECT SupportRepId, SUM(Total) FROM Customer JOIN Invoice... GROUP BY SupportRepId

Come usarlo nel Model:

Estrarre i valori singoli $val_A = diz[A.id]$ e $val_B = diz[B.id]$ e fare le operazioni matematiche $(abs(val_A - val_B) \leq K$ oppure $val_A > val_B)$.

La Traccia d'Esame Simulata:

PUNTO 1:

L'utente seleziona una Nazione (Country) a tendina.

L'applicazione crea un grafo non orientato dove i nodi sono tutti gli Artisti (Artist).

Esiste un arco tra due Artisti se la differenza assoluta tra i brani venduti in quella Nazione dall'Artista A e i brani venduti dall'Artista B è minore o uguale a 5. Se un artista non ha venduto brani, le sue vendite si considerano 0. Il peso dell'arco è la somma delle vendite dei due artisti in quella nazione.

Stampare numero di Nodi, Archi e l'artista con il Grado massimo (numero di archi collegati).

1. Il File DAO.py

Facciamo due query: una semplice per tutti gli artisti (i nodi) e una mirata per le vendite filtrate per Nazione (il dizionario).

codePython

```
from database.DB_connect import DBConnect
```

```
from model.Artist import Artist # Assicurati di avere la classe Artist
```

```
class DAO:
```

```
@staticmethod
```

```
def getCountry():
```

```
    conn = DBConnect.get_connection()
```

```
    cursor = conn.cursor()
```

```
    # Prendo le nazioni senza ripetizioni
```

```
    query = "SELECT DISTINCT Country
```

```
FROM customer
```

```
WHERE Country IS NOT NULL ORDER BY Country"
```

```
    cursor.execute(query)
```

```
    result = []
```

```
    for row in cursor:
```

```
        result.append(row[0])
```

```
    cursor.close()
```

```
    conn.close()
```

```
    return result
```

```
@staticmethod
```

```
def getAllArtists():
```

```
    conn = DBConnect.get_connection()
```

```
    cursor = conn.cursor(dictionary=True)
```

```
    # Prendo tutti gli artisti
```

```
    query = "SELECT * FROM artist"
```

```
    cursor.execute(query)
```

```
    result = []
```

```
    for row in cursor:
```

```
        result.append(Artist(**row))
```

```
    cursor.close()
```

```
    conn.close()
```

```
    return result
```

```
@staticmethod
```

```
def getVenditeByCountry(nazione):
```

```
    conn = DBConnect.get_connection()
```

```
    cursor = conn.cursor(dictionary=True)
```

```
    # TIPOLOGIA 1: DIZIONARIO CONTATORE
```

```
    # Unisco Customer -> Invoice -> InvoiceLine -> Track -> Album -> Artist
```

```
    query = """
```

```
        SELECT ar.ArtistId, COUNT(DISTINCT i.InvoiceId) as vendite
```

```
        FROM customer c, invoice i, invoiceline il, track t, album al, artist ar
```

```
        WHERE c.CustomerId = i.CustomerId
```

```
        AND i.InvoiceId = il.InvoiceId
```

```
        AND il.TrackId = t.TrackId
```

```
        AND t.AlbumId = al.AlbumId
```

```
        AND al.ArtistId = ar.ArtistId
```

```
        AND c.Country = %s
```

```
        GROUP BY ar.ArtistId
```

```
    """
```

```
    cursor.execute(query, (nazione,))
```

```
    # Creo il dizionario { ArtistId : vendite }
```

```
    mappa_vendite = {}
```

```
    for row in cursor:
```

```
        mappa_vendite[row['ArtistId']] = row['vendite']
```

```
    cursor.close()
```

```
    conn.close()
```

```
    return mappa_vendite
```

📁 2. Il File model.py

Questo è il vero cuore dell'esame. Utilizzo .get() per gestire in totale sicurezza gli artisti che hanno venduto 0 copie in quella specifica nazione (quindi non presenti nel dizionario restituito dal DAO).

codePython

```
import networkx as nx

from database.DAO import DAO

class Model:

    def __init__(self):

        self.grafo = nx.Graph() # Grafo NON orientato

    def getCountry(self):

        return DAO.getCountry()

    def buildGraph(self, nazione):

        self.grafo.clear()

        # 1. Aggiungo TUTTI i nodi
        nodi = DAO.getAllArtists()

        for n in nodi:

            self.grafo.add_node(n)

        # 2. Prendo il Dizionario Contatore dal DAO
        # Conterrà roba del tipo: {1: 15, 2: 3, 5: 1 ...}
        mappa_vendite = DAO.getVenditeByCountry(nazione)

        # 3. IL DOPPIO CICLO FOR
        for i in range(len(nodi)):

            for j in range(i+1, len(nodi)):

                u = nodi[i]

                v = nodi[j]

                # LA MAGIA DEL GET:

                Se l'artista non è nel dizionario, metto 0

                vendite_u = mappa_vendite.get(u.ArtistId, 0)

                vendite_v = mappa_vendite.get(v.ArtistId, 0)
```

```
# REGOLA DELLA TRACCIA: differenza vendite <=
5

        diff = abs(vendite_u - vendite_v)

        if diff <= 5:

            # PESO: somma delle vendite

            peso_arco = vendite_u + vendite_v

            # ATTENZIONE: per evitare di collegare
            nodi isolati che hanno entrambi 0

            # spesso conviene aggiungere un
            controllo:

            if peso_arco > 0: # Omettere se la traccia
            accetta pesi 0

                self.grafo.add_edge(u, v,
                weight=peso_arco)

    def getNumNodes(self):

        return self.grafo.number_of_nodes()

    def getNumEdges(self):

        return self.grafo.number_of_edges()

    # Richiesta Punto 1: "Artista con il grado
    massimo"

    def getMaxGrado(self):

        best_artist = None

        max_grado = -1

        for n in self.grafo.nodes():

            grado_corrente = self.grafo.degree(n)

            if grado_corrente > max_grado:

                max_grado = grado_corrente

                best_artist = n

        return best_artist, max_grado
```

📁 3. Il File controller.py

Questo è puramente cosmetico: prende la scelta, la passa al Model e aggiorna l'interfaccia Flet.

```
class Controller:
```

```
    def __init__(self, view, model):
```

```
        self._view = view
```

```
        self._model = model
```

```
    def fillDD(self):
```

```
        # Popolo la tendina delle nazioni all'avvio
```

```
        nazioni = self._model.getCountry()
```

```
        for n in nazioni:
```

```
            self._view._ddNazione.options.append(ft.dropdown.Option(n))
```

```
        self._view.update_page()
```

```
    def handleCreaGrafo(self, e):
```

```
        nazione = self._view._ddNazione.value
```

```
        if nazione is None:
```

```
            self._view.create_alert("Attenzione: devi selezionare una Nazione!")
```

```
            return
```

```
        self._view.txt_result.controls.clear()
```

```
        self._view.txt_result.controls.append(ft.Text("Costruzione del grafo in corso...", color="blue"))
```

```
        self._view.update_page()
```

```
        # 1. Chiamo il Model passando la nazione
```

```
        self._model.buildGraph(nazione)
```

```
        # 2. Rimuovo l'avviso e stampo Nodi e Archi
```

```
        self._view.txt_result.controls.clear()
```

```
        self._view.txt_result.controls.append(ft.Text("Grafo creato con successo!", weight="bold",  
color="green"))
```

```
        self._view.txt_result.controls.append(ft.Text(f"Nodi: {self._model.getNumNodes()}"))
```

```
        self._view.txt_result.controls.append(ft.Text(f"Archi: {self._model.getNumEdges()}"))
```

```
        # 3. Richiedo il nodo con il grado massimo (come da traccia)
```

```
        best_artist, grado = self._model.getMaxGrado()
```

```
        if best_artist is not None:
```

```
            self._view.txt_result.controls.append(  

```

```
ft.Text(f"\nL'artista con il grado maggiore è: {best_artist.Name} (Grado: {grado})", weight="bold")
```

```
            )    self._view.update_page()
```

■ TIPOLOGIA 2: IL Dizionario degli Insiemi (Sets / Condivisione)

Quando usarlo: Ogni volta che leggi la parola **"In comune"**, **"Condiviso"**, o **"Almeno N elementi uguali"**.

Si può fare sia con i set(), sia facendo il lavoro sporco sul db

Possibili richieste all'esame:

1. "Collega due Clienti (Customer) se hanno acquistato **almeno N canzoni in comune**".
2. "Esiste un arco tra l'Artista A e l'Artista B se le loro canzoni compaiono in almeno una Playlist in comune".
3. "Collega due Dipendenti (Employee) se i clienti che gestiscono provengono dallo stesso Country".

TRACCIA:

L'utente seleziona una Nazione (Country). Si crei un grafo non orientato i cui nodi sono i Clienti (Customer) di quella Nazione.

Esiste un arco tra due Clienti se hanno acquistato **almeno 3 brani in comune** (TrackId).

Il peso dell'arco è il numero di brani in comune.

Stampare i 3 archi di peso maggior

📁 1. Il File DAO.py

Facciamo due query: una semplice per tutti gli artisti (i nodi) e una mirata per le vendite filtrate per Nazione (il dizionario).

codePython

```
from database.DB_connect import DBConnect
from model.Artist import Artist # Assicurati di avere la classe Artist
```

class DAO:

```
    @staticmethod
    def getCountry():
        conn = DBConnect.get_connection()
        cursor = conn.cursor()
        # Prendo le nazioni senza ripetizioni
        query = "SELECT DISTINCT Country
FROM customer
WHERE Country IS NOT NULL ORDER BY Country"
        cursor.execute(query)
        result = []
        for row in cursor:
            result.append(row[0])
        cursor.close()
        conn.close()
        return result
```

```
    @staticmethod
    def getCustomersByCountry(nazione):
        conn = DBConnect.get_connection()
        cursor = conn.cursor(dictionary=True)
        query = "SELECT * FROM customer WHERE
Country = %s"
        cursor.execute(query, (nazione,))
        result = []
        for row in cursor:
            result.append(Customer(**row))
        cursor.close()
        conn.close()
        return result
```

```

@staticmethod
def getArchiDiretti(nazione, soglia):
    conn = DBConnect.get_connection()
    cursor = conn.cursor(dictionary=True)

    # LA SUPER QUERY # Trovami ClienteA, ClienteB e quante tracce distinte hanno comprato entrambi
    query = """ SELECT c1.CustomerId as id1, c2.CustomerId as id2, COUNT(DISTINCT il1.TrackId) as
brani_comuni

FROM customer c1, invoice i1, invoiceline il1, customer c2, invoice i2, invoiceline il2

WHERE c1.CustomerId = i1.CustomerId AND i1.InvoiceId = il1.InvoiceId
AND c2.CustomerId = i2.CustomerId AND i2.InvoiceId = il2.InvoiceId

-- CONDIZIONE DI CONDIVISIONE (Stesso Brano)
AND il1.TrackId = il2.TrackId

-- FILTRI DELLA TRACCIA
AND c1.Country = %s AND c2.Country = %s

-- TRUCCO D'ORO: c1.Id < c2.Id per evitare di raddoppiare gli archi (A->B e B->A)
AND c1.CustomerId < c2.CustomerId

GROUP BY c1.CustomerId, c2.CustomerId

-- APPLICO LA SOGLIA GIA' IN SQL (>= 3)
HAVING brani_comuni >= %s """

    cursor.execute(query, (nazione, nazione, soglia))

    # Invece di un dizionario, creo una LISTA DI TUPLE (id1, id2, peso)
    archi = []

    for row in cursor:
        archi.append( (row['id1'], row['id2'], row['brani_comuni']) )

    cursor.close()
    conn.close()

    return archi

```

2. Il File model.py (Senza Doppio Ciclo For!)

```

codePython

import networkx as nx

from database.DAO import DAO

```

```

class Model:

    def __init__(self):
        self.grafo = nx.Graph()
        self.idMap = {}

    def buildGraph(self, nazione):
        self.grafo.clear()
        self.idMap.clear()

        # 1. Aggiungo i NODI (Rimane uguale)
        nodi = DAO.getCustomersByCountry(nazione)

        for n in nodi:
            self.grafo.add_node(n)
            self.idMap[n.CustomerId] = n

        # 2. Prendo gli ARCHI GIA' PRONTI dal DAO
        # (Passo la nazione e la soglia N=3)
        lista_archi = DAO.getArchiDiretti(nazione, 3)

        # 3. UN SOLO CICLO FOR (Inserisco nel grafo)
        for arco in lista_archi:
            # L'arco è una tupla: (id_cliente1, id_cliente2, peso)

            id_u = arco[0]
            id_v = arco[1]
            peso = arco[2]

            # Recupero l'oggetto Cliente corrispondente
            # dalla mia idMap

            u = self.idMap[id_u]
            v = self.idMap[id_v]

            # Aggiungo l'arco al grafo
            self.grafo.add_edge(u, v, weight=peso)

        # Lambda per prendere i 3 archi di peso maggiore (richiesta del Controller)
        def getTop3Archi(self):
            lista_archi = []
            for u, v, data in self.grafo.edges(data=True):
                lista_archi.append((u, v, data['weight']))
            lista_archi.sort(key=lambda x: x[2], reverse=True)
            return lista_archi[:3]

```

2. Il File Controller.py (Senza Doppio Ciclo For!)

```
import flet as ft
class Controller:
    def __init__(self, view, model):
        self._view = view
        self._model = model

    # 1. Popola la tendina all'avvio del programma
    def fillDD(self):
        nazioni = self._model.getCountry()
        for n in nazioni:
            self._view._ddNazione.options.append(ft.dropdown.Option(n))
        self._view.update_page()

    # 2. Crea il grafo quando si preme il bottone
    def handleCreaGrafo(self, e):
        nazione = self._view._ddNazione.value

        # Validazione input
        if nazione is None:
            self._view.create_alert("Attenzione: devi selezionare una Nazione!")
            return

        # Pulisco l'interfaccia
        self._view.txt_result.controls.clear()
        self._view.update_page()

        # CHIAMO IL MODEL PER CREARE IL GRAFO
        self._model.buildGraph(nazione)

        # Stampo le informazioni base (Nodi e Archi)
        self._view.txt_result.controls.append(ft.Text("Grafo creato con successo!", weight="bold",
color="green"))
        self._view.txt_result.controls.append(ft.Text(f"Numero di nodi: {self._model.getNumNodes()}"))
        self._view.txt_result.controls.append(ft.Text(f"Numero di archi: {self._model.getNumEdges()}"))

        top3 = self._model.getTop3Archi() # Il model ci restituisce una lista ordinata

        if not top3:
            self._view.txt_result.controls.append(ft.Text("\nNessun arco presente nel grafo.", color="red"))
        else:
            self._view.txt_result.controls.append(ft.Text("\nI 3 archi con il maggior numero di brani in
comune sono:", weight="bold"))

            for u, v, peso in top3:
                testo_arco = f"{u.FirstName} {u.LastName} <--> {v.FirstName} {v.LastName} : {peso} brani"
                self._view.txt_result.controls.append(ft.Text(testo_arco))

        self._view.update_page()
```


TIPOLOGIA 3: Il Dizionario Semplice (Soglia o Valore Diretto)

Struttura: { ID_NODO : Attributo } (es. { 1: "USA" } o { 1: 5000 }).

Quando usarlo: Quando il dato necessario per collegare due nodi si trova già nella tabella dei nodi, ma ti serve estrarlo velocemente per applicare un filtro. Spesso questo si può fare senza nemmeno il dizionario (leggendo l'attributo dell'oggetto), ma in alcuni casi complessi è comodo.

Possibili richieste all'esame:

1. "Collega due Nodi (Track) se la differenza della loro durata in millisecondi è $\leq X$ ".
 - o Nota: Qui il dizionario non serve se hai già salvato i Milliseconds nell'oggetto Track. Basta fare:

codePython

```
if abs(A.Milliseconds - B.Milliseconds) <= X:
```

2. "Collega due Clienti se appartengono alla stessa nazione (Country)".
 - o Stesso discorso:

codePython

```
if A.Country == B.Country:
```

Quando creare gli oggetti e quando no

- Il Testo dice "I Nodi del grafo sono i [Tabella]" → Crea gli oggetti Python (Customer(), Track(), Artist()).
- Il Testo ti chiede di popolare una Tendina (Dropdown) con Nomi o Nazioni → Restituisci semplici Stringhe ("Rock", "Italy").
- Il Testo ti chiede di fare una somma, un count o trovare elementi in comune → Restituisci Dizionari di Numeri ({id: 5}) o Set di ID ({id: {1,2,3}}).
- Il Testo chiede di ordinare "I Top 5" → Restituisci Liste di Tuple [(Oggetto_Nodo, Punteggio)] e fai il .sort(lambda).

CONTENUTI DI OGNI ESAME

12-01-2026- grafo semplice e pesato

- da due menù a tendina due anni che definiscono un range di interesse
- nodi : tutti i costruttori che hanno partecipato ad almeno una gara nel corso del periodo selezionato
- archi: se i due costruttori hanno condiviso almeno un pilota durante il periodo selezionato
- peso : numero di piloti che hanno corso per entrambi i costruttori nel periodo considerato
- 3 archi di peso maggiore, num comp connesse, comp conn con peso maggiore e tutti i nodi secondo il grado dei nodi

10-11-2025- grafo orientato e pesato

- Nodi : gli ordini che sono stati effettuati nello store selezionato
- Archi : se sono stati ordinati a una distanza temporale di massimo K giorni (valore K incluso). L'arco va sempre dall'ordine effettuato in data precedente verso data successiva
- Peso: $(\text{numero_totale_oggetti_nei_due_ordini}) / (\text{giorni_tra_ordini})$
- cinque archi di peso maggiore, "Cerca Percorso Massimo", si visualizzi il cammino più lungo partendo da un nodo

SIMULAZIONE1- un grafo orientato e pesato

- dd: genere
- nodi: artisti che possiedono almeno un brano (Track) appartenente al genere selezionato.
- Archi: se almeno un cliente ha acquistato brani di entrambi gli artisti, con verso da A verso B se la popolarità di A è maggiore della popolarità di B. In caso i nodi A e B abbiano la stessa popolarità, aggiungere due archi in entrambi i versi.
- Peso: somma delle popolarità
- l'artista con maggiore influenza-> peso archi uscenti – peso archi entranti, 5 archi con peso maggiore, in ordine decrescente.

SIMULAZIONE 2- un grafo NON orientato ma pesato

- dd: un range di rating
- nodi : attori che hanno recitato nei film che hanno ricevuto una valutazione nel range definito
- archi: due attori se hanno recitato almeno in uno stesso film
- peso: somma degli incassi dei film in comune.
- 5 archi di pesi maggiore, num comp connesse, comp conn con peso maggiore

SIMAI1-grafo non orientato e pesato

- dd: mediatype
- nodi: artisti che possiedono almeno un brano pubblicato nel formato MediaType selezionato
- archi : se esiste almeno una fattura che contiene contemporaneamente brani di A e brani di B
- peso: numero di fatture distinte in cui compaiono brani di entrambi gli artisti.
- I due artisti collegati dall'arco con il peso massimo, L'artista (o gli artisti in caso di parimerito) con il grado massimo, ovvero l'artista connesso al maggior numero di altri artisti

SIMAI2-grafo non orientato e pesato

- dd: playlist, k
- nodi: le tracce contenute nella Playlist selezionata
- archi: se la differenza assoluta tra le loro durate in millisecondi è minore o uguale a **k**
- peso: differenza assoluta in Bytes tra la Traccia A e la Traccia B.
- I due nodi collegati dall'arco con il peso MINIMO

SIMAI3- grafo semplice, non orientato e pesato

- DD: due Anni che definiscono un range di interesse .I menù dovranno essere riempiti interrogando il database per ottenere gli anni in cui è stata emessa almeno una fattura (Invoice.InvoiceDate).
- Nodi: TUTTI gli artisti (Artist) presenti nel database – memorizzare le proprietà dell'artista (colonne tabella Artist); o ii) le vendite delle sue tracce avvenute negli anni selezionati (2 query separate)
- Archi : se entrambi gli artisti hanno venduto almeno una traccia nel range di anni selezionato
- Peso: somma del numero totale di tracce vendute nei vari anni per i due artisti collegati
- componente connessa di dimensione maggiore, e stamparne tutti i nodi, ordinati in senso decrescente in base al peso massimo degli archi incidenti su quel nodo.

SIMAI4- grafo diretto e pesato

- dd: genere- 2 datePickers
- nodi: tutti i brani del genere selezionato dall'utente
- archi: solo se entrambi i brani sono stati venduti almeno una volta nel range di date selezionato L'arco è uscente dal nodo con numero di vendite maggiore ed entrante nel nodo con numero di vendite minore. In caso di parità, si inseriscano entrambi gli archi.
- Peso: somma delle vendite dei due brani nel range considerato
- i 5 brani con il punteggio più alto, ovvero i nodi la cui somma dei pesi degli archi uscenti meno la somma dei pesi degli archi entranti è massima.

SIMAI5- grafo semplice, non orientato e pesato

- dd: Country
- nodi : tutti i Clienti (Customer) che risiedono nella nazione selezionata
- archi: se la differenza in valore assoluto del totale speso dai due clienti è inferiore o uguale a X
- peso: somma dei soldi spesi dal Cliente A e dal Cliente B
- I 3 Archi di peso maggiore

10/07/2025- grafo diretto e pesato

- dd e 2 datePickers
- nodi: tutti i prodotti della categoria selezionata
- archi: se entrambi i prodotti sono stati venduti almeno una volta nel range selezionato . L'arco è uscente dal nodo con numero di vendite maggiore ed entrante nel nodo con numero di vendite minore. In caso di parità, si inseriscano entrambi gli archi.
- Peso: somma delle vendite dei prodotti nel range considerato
- 5 prodotti più venduti, ovvero i nodi la cui somma dei pesi degli archi uscenti meno la somma dei pesi degli archi entranti è massima.

15/09/2025- grafo semplice, non orientato e pesato

- dd: 2 anni
- nodi: tutti i piloti che hanno disputato almeno una gara nel corso del periodo selezionato(solo se hanno tagliato il traguardo)
- archi: solo se i due piloti hanno corso (i.e. abbiano partecipato almeno ad una gara) come compagni di squadra almeno per un anno nel range selezionato
- peso: somma del numero di gare che i piloti hanno corso come compagni di squadra
- 3 archi di peso maggiore, num comp connesse, comp conn con peso maggiore e tutti i nodi secondo il grado dei nodi

26-06-2025-grafo semplice e pesato

- dd: 2 anni
- nodi: tutti i circuiti su cui è mai stato disputato un gran premio di F1 indipendentemente dagli anni selezionati- memorizzare: i) le proprietà del circuito (colonne della tabella circuits); ii) i risultati dei gran premi tenuti in tale circuito negli anni selezionati (2 query)
- archi: se e solo se entrambi i circuiti hanno ospitato la Formula1 almeno per un anno nel range selezionato
- peso: somma del numero di piloti che hanno correttamente tagliato il traguardo nei vari anni per i due circuiti collegati
- componente connessa di dimensione maggiore, e stamparne tutti i nodi, ordinati in senso decrescente di peso massimo degli archi incidenti.

13/01/2025- grafo semplice (non orientato) e pesato

- dd in ordine alfabetico
- nodi: tutte le Classificazioni contenute nel database
- archi: se è solo se tra di loro esiste una interazione
- peso: somma di tutti i cromosomi
- Gli archi, con il corrispettivo peso, ordinati in senso crescente di peso

04/07/2024- grafo orientato e non pesato

- dd in ordine decrescente, dd relativo all'anno
- nodi: tutti gli avvistamenti che siano avvenuti nell'anno selezionato dall'utente e con la shape desiderata.
- Archi: se e solo se tali avvistamenti sono avvenuti nello stesso stato- L'arco è uscente dall'avvistamento che è avvenuto temporalmente prima ed entrante nell'avvistamento avvenuto dopo.
- numero di componenti debolmente connesse, identificare la componente connessa di dimensione maggiore, e stamparne i nodi – includendo il dettaglio della città in cui è avvenuto l'avvistamento e la data

18/07/2024- grafo diretto e pesato

- dd: min e max
- nodi: tutti i geni contenuti nella tabella genes il cui Chromosoma appartiene a min e max
- archi: se e solo se i due geni hanno la stessa Localizzazione, GeneID diverso, ed esiste una interazione tra di loro
- peso: 'indice di correlazione dell'interazione fra i due geni
- 5 nodi col maggiore numero di archi uscenti col numero di archi uscenti ed il peso complessivo di questi archi (la somma dei loro pesi). I nodi devono essere stampati in ordine decrescente per numero di archi uscenti.